Day 7 Revision Sheet

Oct 26, 2026 | Python: loops revision, lists, tuples | Git: log and commit ID | Smooth (BCA, learning on phone + Colab)

1. Loops revision: key ideas

- **range(start, stop, step):** the step is the amount ADDED to the index each time (step 2 skips one number, not two). Stop is excluded. `range(1,6,2)` gives 1, 3, 5 because the next value, 7, is past stop 6.
- **Include n by writing n+1:** `range(1,101)` gives 1 to 100. `range(1,100)` gave the same answer for 'divisible by 15' only by luck (100 is not a multiple of 15). For 'divisible by 10' it stops at 90.
- **while + continue:** put the update (`i += 1`) BEFORE `continue`. If it comes after, continue skips it, i never changes and the loop never ends. There is no range in a while loop.
- **Divisible by two numbers:** use the LCM. 3 and 5 -> `i%15==0`. 4 and 6 -> 12, not 24. LCM = a*b / GCD (4*6/2 = 12). The product equals the LCM only when GCD is 1.
- **Max pattern:** `highest = marks[0]`, then update if `mark > highest`. Starting at 0 breaks on negatives: [-5,-2,-9] gives 0, a number that is not in the list. > keeps the first max seen, >= takes the last (matters for the index).
- **Never name a variable max, sum, list, str, len.** `max = marks[0]` replaces the built-in, and a later `max([1,2])` raises TypeError: 'int' object is not callable.

2. Lists in depth

2.1 Index and negative index

- Index = offset (steps from the first item), not position. `nums = [10,20,30,40,50]`: nums[0] = 10, nums[1] = 20, nums[-1] = 50. Excel counts positions (A1 is 1st); Python counts offsets.
- Valid indexes for n items: **-n to n-1**. `nums[5]` and `nums[-6]` both raise **IndexError: list index out of range**.
- Convert a negative index first: **n + index**. `nums[-2]` -> 5 + (-2) = 3 -> value 40. Always carry it through to the value.

2.2 Slicing [start:stop:step]

- Same three numbers and rules as range(): start included, stop excluded, step added to the index. Count of items = stop - start (when both are inside the list).
- Missing parts with a positive step: start = 0, stop = len. `nums[:3]` -> [10,20,30]. `nums[2:]` -> [30,40,50]. `nums[::2]` -> [10,30,50] (indexes 0, 2, 4).
- **Direction is decided ONLY by the sign of the step.** A negative start or stop only chooses where to stand. `nums[-3:]` -> [30,40,50] (walks forward, no step given). `nums[::-1]` -> [50,40,30,20,10]: a negative step flips the defaults (start at the last index, stop past the first).
- `nums[:-2]` -> [10,20,30]: -2 becomes index 3 (value 40), stop is excluded, so it ends at index 2 (value 30). `nums[:-2] + nums[-2:]` rebuilds the whole list.
- A slice is a NEW list. Out-of-range slices never raise IndexError (`nums[2:100]` -> [30,40,50]) while a single index does. `nums[-0:]` is the whole list because -0 is 0.

2.3 List methods: what changes, what comes back

Method	What it does	Changes original?	Returns	Watch out
append(x)	adds x at the end	yes	None	`x = nums.append(5)` makes x None
insert(i,x)	puts x at index i, shifts the rest right	yes	None	nothing is replaced
remove(x)	deletes the first item equal to x (by VALUE)	yes	None	only one match; ValueError if x absent
pop() / pop(i)	removes the last item / the item at i	yes	the removed item	pop() takes from the END; IndexError on empty list
sort()	sorts the list itself	yes	None	`nums = nums.sort()` leaves None; lists only
sorted(x)	builds a new sorted list	no	new list	works on any iterable; `reverse=True` for high to low

- **Rule:** in-place methods return None. `nums + [60]` builds a new list and leaves nums alone (I checked: nums stayed 5 items, the new list had 6).
- **Traps:** removing items while looping over the same list skips items (`[20,20,30]` loop-remove 20 leaves [20,30]); loop over a copy `nums[:]` instead. `pop(0)` and `insert(0,x)` are slow on big lists. Sorting is stable (ties keep their order).
- **Excel bridge:** append = type in the next empty cell of the column. remove = Find first match, then delete (not Replace All). sort() sorts the column itself; sorted() sorts a copy on another sheet.

2.4 Tuples

- A tuple can be read and sliced like a list but **cannot be changed**: `t[1] = 99` -> **TypeError: 'tuple' object does not support item assignment**. Strings are immutable too: `s[0] = 'j'` -> TypeError: 'str' object does not support item assignment. Say 'immutable' when asked for the property.
- To 'change' a tuple, build a NEW one: `(record[0], 95)` or `t + (99,)`. The original stays as it was.
- **The comma makes the tuple, not the brackets.** `(5)` is an int, `(5,)` is a tuple (`<class 'tuple'>`).
- **Immutable does not freeze what is inside.** `t = (1, [2,3])`: slot 0 holds 1, slot 1 holds a LIST. `t[1].append(4)` works -> (1,[2,3,4]) because the tuple still holds the same list. `t[1] = [9]` fails with TypeError.

- Excel bridge: a tuple is a sheet with locked cells. You can read every cell, but you edit by making a new sheet.

3. Git: log and commit ID

- `git log` shows COMMITTED history only. Staged files show in `git status`. `git log --oneline` gives one line per commit.
- A commit ID is a 40-character hash; the short form is 7 characters (e.g. c6cd6c0). It identifies ONE exact commit snapshot with its history, not the repo. Change anything and the ID changes.
- Three commits all named 'third commit' make the log useless. Write what changed: 'Add loops practice'.

4. My mistakes and the fix

What I said / did	Correct version
Step = 'how many to skip'	Step is the amount added to the index each time.
Continue bug: 'range not updating'	A while loop has no range. The loop variable never changes, so it never ends. Update before continue.
Max starts 'from index 0'	Start from the first VALUE marks[0]. Starting at 0 fails on all-negative lists.
'Index out of bound error'	The exact name is IndexError: list index out of range.
`nums[-3:]` walked backward	No step means +1, so it walks forward: [30,40,50]. Only the step sign gives direction.
`nums[:-2]`: 'second from last is 30'	-2 -> index 3 -> 40. Stop is excluded, so the slice ends at 30.
`x = nums.append(50)` printed a list	append returns None. nums changed in place and has 5 items.
'Item assignment error'	TypeError: 'tuple' object does not support item assignment. Name it exactly.
`type((5,))` answered int	The comma makes a tuple. My reason and my output contradicted each other.
Slot 1 of (1,[2,3]) 'holds an int'	Slot 0 holds 1. Slot 1 holds the list.
pop() 'removes from the start'	pop() removes from the END. It is the mirror of append().
Named a variable max	It replaces the built-in: TypeError 'int' object is not callable.
`except exception`	Class names are capitalised: Exception. Lowercase gives NameError.
range(1,100) 'gives the same output'	Luck. It misses 100 for 'divisible by 10'. To include n, write n+1.
LCM of 4 and 6 'by multiplying'	12, not 24. The product counts the shared factor 2 twice.
git log shows staged files; commit ID identifies the repo	Log shows commits only. A commit ID identifies one commit.

5. How I improved today

- I explained `nums[::-1]` using the negative STEP, not a negative index, and began converting -2 to index 3 before tracing.
- I found the multiple-of-15 shortcut myself, and my sort/sorted and remove/pop outputs were right on the first try.
- I ran code before trusting it (max name clash, tuple record) and renamed `max` to `highest`.

6. Practice to retry (no solutions)

1. `temps = [31, 28, 35, 35, 29]`: find the highest and how many times it appears without max(). Then say what your code returns for [-4, -9, -1].
2. Print `words = ['pen','book','bag']` in reverse WITHOUT changing the original, two different ways. Say which one returns None.
3. `nums = [5,10,15,20,25,30]`: predict `nums[-4:]`, `nums[:-4]`, `nums[1::2]`, `nums[::-2]`. Convert every negative number to a positive index first.
4. `a = [3,1,2]; b = a.sort(); c = sorted(a, reverse=True)`. Predict a, b and c, line by line, before running.
5. Print the numbers from 1 to 200 divisible by 4 and 6 using ONE condition. Predict how many print and why.
6. `record = ('Ravi', 72)`: set the score to 80 without editing the tuple. For `t = (1,[2,3])`, which fails: `t[1] = [0]` or `t[1].append(0)`? Name the error.

7. Recall questions (answer from memory)

1. Why must the update line come before continue in a while loop?
2. What decides the direction of a slice, and what do negative start and stop do?
3. Which list methods return None, and which return the removed item?
4. Give two differences between sort() and sorted().
5. Why is `(5,)` a tuple and `(5)` an int?
6. What does a commit ID identify, and what does git log show?
7. For n items, what is the valid index range, and how do you convert -2?

8. Open items

- Re-test weak spots: step wording, naming the exact error, saying 'stop is excluded' in every reason, pop() from the END.
- Commit the Day 7 README entry as its own commit with a message that says what changed.
- Not yet taught: list comprehension, safe removal while looping, deque. Teach when the plan reaches them.
- Oct 28 night tasks are next.